

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset $\text{\LaTeX}$ solutions.

1.a

$\text{Rain} \rightarrow \text{WetGrass}$ and $\text{Sprinkler} \rightarrow \text{WetGrass}$. There is no edge between Rain and Sprinkler, since neither one causes the other.

1.b

The joint is the product of each variable's local conditional probability given its parents:

$$\mathbb{P}(\text{Rain} = r, \text{Sprinkler} = s, \text{WetGrass} = w) = p(r) p(s) p(w \mid r, s).$$

1.c

Sum the joint over both values of Rain and both values of Sprinkler:

$$\mathbb{P}(\text{WetGrass} = w) = \sum_r \sum_s p(r) p(s) p(w \mid r, s).$$

1.d

The grass is wet with probability 0.9 when it rained and the sprinklers were on, so it stays dry with probability $1 - 0.9 = 0.1$:

$$\begin{aligned} \mathbb{P}(\text{Rain} = 1, \text{Sprinkler} = 1, \text{WetGrass} = 0) &= p(\text{Rain} = 1) p(\text{Sprinkler} = 1) p(\text{WetGrass} = 0 \mid \text{Rain} = 1, \text{Sprinkler} = 1) \\ &= 0.2 \times 0.5 \times 0.1 = 0.01. \end{aligned}$$

1.e

- (a) True.
- (b) False.
- (c) True.

2.d

One run of `forward_sampling` with `mutation_rate = 0.1` and `genome_length = 10`:

Species	Genome
Thomas bayus	TTCCATTCCC
Humblus studentus	TGCCATTCCCT
Aryamus bayus	TTCCATTCCC
Kenius bayus	TTCCATTCCC

2.g

Estimates of $\mathbb{P}(\text{ThomasBayus} = \text{AAAC} \mid \text{KeniusBayus} = \text{AAAA})$, from `test_gibbs_vs_rejection` with `mutation_rate = 0.1`, `genome_length = 4`, seed 0:

Method	Steps/Iterations	Estimated Posterior
Rejection sampling	100	0.0000
Rejection sampling	10,000	0.0667
Gibbs sampling	100	0.0900
Gibbs sampling	10,000	0.0332
Exact	—	0.0335

3.a

$Y \rightarrow A_1$, $Y \rightarrow A_2$ and $Y \rightarrow A_3$. The true label Y is what each annotator's claimed label depends on, and the annotators do not influence one another, so there are no edges between them.

3.b

A perfect annotator always gives the true label.

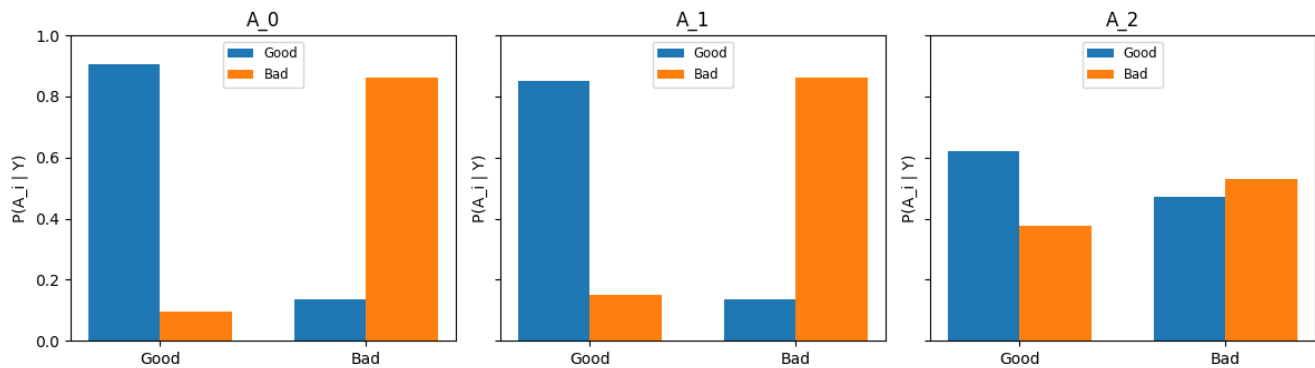
Y	$p(A_1 = \text{good} \mid Y)$	$p(A_1 = \text{bad} \mid Y)$
good	1	0
bad	0	1

3.c

A random annotator ignores the true label, so both rows are the same.

Y	$p(A_1 = \text{good} \mid Y)$	$p(A_1 = \text{bad} \mid Y)$
good	0.5	0.5
bad	0.5	0.5

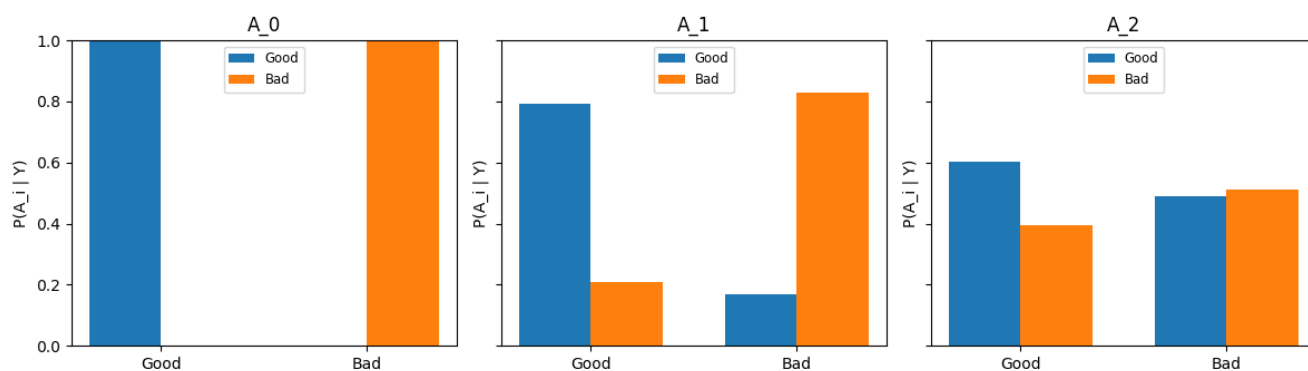
3.g



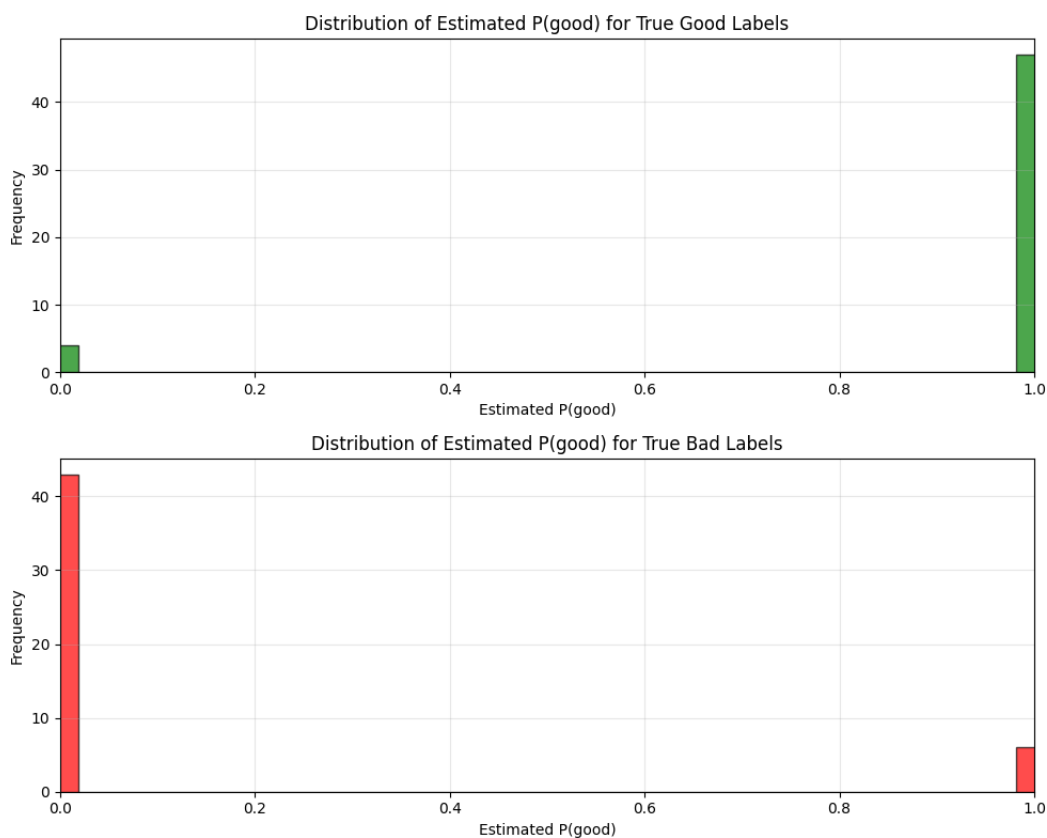
A_2 is the least trustworthy. Its answers are close to a coin flip whichever the true label is: when the conversation is good it says good about 62% of the time, and when it is bad it only says bad about 53% of the time, so its table looks much more like the random annotator from 3c than the perfect one from 3b. A_0 and A_1 both get the label right around 85 to 90% of the time either way.

4.d

Annotator tables after 100 iterations of EM on the unlabelled annotations:



Estimated $P(\text{good})$ for each data point, split by its true label:



4.e

Only partly. About 90 of the 100 data points end up on the right side: most of the truly good ones get an estimated $P(\text{good})$ near 1 and most of the truly bad ones near 0, with a handful on the wrong side in each. But the annotator plot shows EM has turned A_0 into a perfect annotator, so what it really learned is A_0 's labels, treated as the truth. That works reasonably well here because A_0 was the most accurate annotator, right about 90% of the time when trained with the true labels in 3g, and the data points EM gets wrong are the ones A_0 got wrong.

5.a

Claude Code is built for writing code, mostly automating programming tasks like adding features, writing tests and building small applications, so it matters most for software developers, especially front-end and web developers. Anthropic's own study of coding conversations found that 79% of Claude Code conversations were automation rather than augmentation, against 49% on Claude.ai, with building user interfaces the most common use, and it suggested that jobs centred on simple applications and user interfaces may face disruption sooner than backend work ([Anthropic Economic Index, April 2025](#)). Startups were the main early adopters, with enterprises lagging behind. Upstream, the product also depends on human labour that is easy to miss: the paid crowd workers who rate responses and test the model for safety, described in the Opus 5 system card, and the developers and writers whose code and text make up the training data, which is what the copyright suits over training data are about.

5.b

Claude Code is more likely to replace entry-level work and augment experienced developers than to replace developers outright. Stanford's *Canaries in the Coal Mine* study finds employment for 22 to 25 year olds in AI-exposed jobs, software development included, is now 19% below where it should be while experienced workers show no gap, and the losses are concentrated in jobs where AI automates rather than assists, which fits Claude Code being used mostly for automation ([Stanford Digital Economy Lab](#)). For workers the risk is that if companies stop hiring juniors, there is no route to becoming a senior, so the pipeline of experienced developers dries up. For employers the risk is code that nobody fully understands, since code can pass its tests while still being subtly wrong, which I ran into several times in this course. Layoffs blamed on AI are real, but those are companies' stated reasons, so it is hard to tell how many jobs were actually replaced.

5.c

Claude Code favours English-speaking, higher-income users. Plans run from about \$17 a month for Pro to \$100–200 for Max, so wealthier users and companies get the stronger models and higher usage limits, and the usage policy cuts off access entirely in unsupported regions, so lower-income users and countries benefit least. Performance differs too, not just access: the Opus 5 system card admits output quality varies by language, but its multilingual results give only a single average across 42 languages (92.5%) with no per-language breakdown, and nothing for coding, so a developer prompting in a language other than English may get worse code from the same subscription without anyone being able to see it, which widens the gap even when access is equal. To monitor this, Anthropic could publish Claude Code's task success and error rates broken down by prompt language and by plan tier.

5.d

Instead of giving everyone full automation up front, companies could scale how much Claude Code does with how experienced the developer is. Juniors would get a low level of automation, with Claude Code set up to explain and help them understand rather than write finished code, and high human oversight, approving each action and having their changes reviewed. As they gain experience, oversight would drop and automation would rise, until they work the way senior developers do now. Claude Code already supports this, since its permission modes range from asking before every action to working on its own. This keeps companies hiring juniors, since a junior with Claude Code is still productive, while making sure they actually learn, so the pipeline from 5b does not dry up and there are still people who understand the code being shipped.

6.a

Claude Code is Anthropic's agentic coding tool. It runs in the terminal or an IDE, reads a codebase, edits files, runs commands and makes commits and pull requests, working towards a goal on its own and asking for approval according to its permission mode. Its strengths showed in my own use: it added a feature and unit tests to an existing investment backtester, built an application from scratch, and worked as a tutor through this course. Anthropic markets it mostly through developer channels rather than hype, and publishes a system card for each model, a plain-language usage policy and regular threat intelligence reports. Its weaknesses are mostly about what users cannot see or control. The training default for consumer accounts is never stated, the Claude Code data page is hard to find, and anything sent through `/feedback` is kept for five years. Individual users get no copyright indemnity while businesses do, and support is private and asynchronous with no public record of what was reported or fixed. Its fairness testing covers only the chat model, with multilingual results reported as a single average. And because it is used mostly for automation, it puts entry-level developers at the most risk, while its code can pass tests and still be subtly wrong.

On privacy and accountability, I would state the training default plainly and show the current setting, and what it means for retention, the first time Claude Code is launched. I would add a per-session view showing whether each session was kept for training and when it will be deleted, and tie the retention of `/feedback` reports to the user's training setting rather than a flat five years. I would also make reported issues public, with what Anthropic did about each one and a way for users to vote on what gets fixed first, so users have real influence over the product.

On fairness and jobs, I would publish Claude Code's own task success and error rates broken down by prompt language and plan tier, so performance gaps can actually be seen and tracked. For junior developers, I would offer a mode that starts with low automation and high oversight, explaining rather than writing finished code and asking before each action, and gradually hands over more automation as the developer gains experience. Finally, I would extend copyright protection to individual users, or at least flag outputs that closely match known licensed code, so the risk is not left entirely on the people least able to carry it.

6.b

Right now Anthropic decides how Claude Code changes, with its engineers building it and its executives deciding what ships, and the only outside groups with real power are courts and paying businesses: in the music publishers' case Anthropic agreed to keep its copyright guardrails in place while the case continues, and the commercial terms give businesses protections that individual users do not get. Individual users can report problems through `/bug` and the GitHub tracker, but that path is private and slow, with over 5,000 open issues, so in practice they have almost no say. I think users should have real influence too, since they are the ones whose code, data and jobs are affected, and they are the first to see its problems. One way to shift that would be a public record of reported issues and what Anthropic did about them, with users able to vote on what gets fixed first, so their feedback has visible weight instead of disappearing into a private queue.